```
    output.close
    File.delete(file_name)
    raise
  end
end
```

Because we specified StandardError in the rescue line, that clause will handle exceptions of class StandardError and all of its children, which means that we won't catch Ruby internal errors, which is fine. In the error-handling block, we report the error, close and delete the output file, and then re-raise the exception. It's also worth noting that the close and delete calls could also raise exceptions, and those exceptions are not caught in the code above. We'll see a partial fix for that in a moment.

As matter of style, the rescue statement is outdented to the level of the begin/end block.

When an exception is raised, independent of any subsequent exception handling, Ruby places a reference to the associated exception object into the global variable $! (the exclamation point presumably mirroring our surprise that any of *our* code could cause errors). In the previous example, we used the $! variable to format our error message.

After closing and deleting the file, we call raise with no parameters, which reraises the exception that is currently stored in $!. This is a useful technique, because it allows you to write code that filters exceptions, passing on those you can't handle to higher levels. It's almost like implementing an inheritance hierarchy for error processing.

You can have multiple rescue clauses in a method or begin block, and each rescue clause can specify multiple exceptions to catch. At the end of each rescue clause, you can give Ruby the name of a local variable to receive the matched exception. Most people find this more readable than using $! all over the place:

```
begin
  eval string
rescue SyntaxError, NameError => e
  print "String doesn't compile: " + e
rescue StandardError => e
  print "Error running script: " + e
end
```

How does Ruby decide which rescue clause to execute? It turns out that the processing is similar to that used by the case statement. For each rescue clause in the begin block, Ruby compares the raised exception against each of the parameters in turn. If the raised exception matches a parameter, Ruby executes the body of the rescue and stops looking. The match is made using *parameter* === $!.

This means that the match will succeed if the exception named in the rescue clause is the same as or a superclass of the type of the currently thrown exception. This comparison happens because exceptions are classes, and classes in turn are kinds of Module. The === method is defined for modules, returning true if the class of the operand is the same as or is a descendant of the receiver. If you write a rescue clause with no parameter list, the parameter defaults to StandardError, so technically our declaration of StandardError in the earlier code is redundant.